

Trabalho 2

ENTREGA: 11/06 A ausência ou atraso resulta em nota zero.

Tema: Emulador de uma ISA Didática

O objetivo deste trabalho é desenvolver um emulador de uma arquitetura didática simples. O programa deverá ler uma sequência de instruções, executar cada instrução passo a passo e mostrar o estado interno da máquina durante a execução. O emulador poderá ser desenvolvido em qualquer linguagem de programação ou outra ferramenta equivalente.

1 Descrição Geral da Arquitetura

A arquitetura emulada deverá seguir um modelo *load-store* simplificado. Nesse modelo, as operações aritméticas são realizadas apenas entre registradores, enquanto o acesso à memória ocorre apenas por meio das instruções **LOAD** e **STORE**.

O emulador deverá possuir, no mínimo:

- um **PC**, responsável por indicar a próxima instrução a ser executada;
- um **banco de registradores**, contendo pelo menos R0, R1, R2 e R3;
- uma **memória de dados**, com pelo menos 32 posições;
- uma **memória de instruções**, contendo o programa carregado;
- um ciclo básico de busca, decodificação e execução.

O registrador R0 deverá representar sempre o valor zero, de forma semelhante ao registrador **x0** da arquitetura RISC-V.

2 Conjunto de Instruções

O emulador deverá implementar obrigatoriamente as seguintes instruções:

Instrução	Operação
LOAD rd, offset(rs1)	$\text{Reg}[\text{rd}] = \text{Mem}[\text{Reg}[\text{rs1}] + \text{offset}]$
STORE rs2, offset(rs1)	$\text{Mem}[\text{Reg}[\text{rs1}] + \text{offset}] = \text{Reg}[\text{rs2}]$
ADD rd, rs1, rs2	$\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] + \text{Reg}[\text{rs2}]$
SUB rd, rs1, rs2	$\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] - \text{Reg}[\text{rs2}]$
HALT	Encerra a execução do programa

As instruções devem ser escritas em formato textual, semelhante a assembly. O emulador deverá interpretar cada linha do programa, identificar a instrução e executar o comportamento correspondente.

3 Funcionamento Esperado

Durante a execução, o emulador deverá realizar as seguintes etapas para cada instrução:

1. buscar a instrução indicada pelo PC;
2. decodificar a instrução e seus operandos;
3. executar a operação correspondente;
4. atualizar registradores ou memória, quando necessário;
5. atualizar o PC;
6. exibir o estado atual da máquina.

A execução deverá continuar até que a instrução **HALT** seja encontrada.

4 Saída do Emulador

A cada instrução executada, o emulador deverá exibir, no mínimo:

- valor atual do PC;
- instrução executada;
- valores dos registradores;
- alterações realizadas na memória, quando houver.

Um exemplo simplificado de saída seria:

PC	Instrução	Estado após execução
0	LOAD R1, 10(R0)	R1 = 7
1	LOAD R2, 11(R0)	R2 = 5
2	ADD R3, R1, R2	R3 = 12
3	STORE R3, 12(R0)	Mem[12] = 12
4	HALT	Execução encerrada

5 Programa de Teste Obrigatório

O grupo deverá demonstrar o emulador executando, no mínimo, o programa abaixo:

Instrução	Comportamento esperado
LOAD R1, 10(R0)	R1 = Mem[10]
LOAD R2, 11(R0)	R2 = Mem[11]
ADD R3, R1, R2	R3 = R1 + R2
SUB R4, R3, R1	R4 = R3 - R1
STORE R3, 12(R0)	Mem[12] = R3
STORE R4, 13(R0)	Mem[13] = R4
HALT	Encerra a execução

Considerando Mem[10] = 7 e Mem[11] = 5, ao final da execução espera-se que Mem[12] = 12 e Mem[13] = 5.

6 Entregáveis

O grupo deverá entregar:

1. código-fonte do emulador;
2. arquivo executável, link ou instruções claras de execução;
3. pelo menos dois programas de teste;
4. apresentação em sala com demonstração do emulador funcionando.

7 Funcionalidades Opcionais

As funcionalidades abaixo não são obrigatórias, mas podem melhorar a qualidade do trabalho:

- carregar o programa a partir de um arquivo `.asm`;
- implementar instruções de desvio, como `JMP` ou `BEQ`;
- implementar modo de execução interativo, avançando uma instrução por vez;
- exibir a memória e os registradores em formato de tabela;
- validar erros de sintaxe nas instruções.